Laborbuch 3

Praktikum 3

Wir empfehlen Ihnen die Bearbeitung auf Papier und die anschließende Übertragung hierher. Mit dem Visual Studio Code Plugin für "draw.io" können Sie die PNG-Dateien direkt bearbeiten!

von: **Levi Blumenwitz**

CircuitVerse Arbeitsbuch: https://circuitverse.cs.hm.edu/users/203/projects/praktikum-3-f729fc65-5b53-41e4-8a0c-7fa2edeb16a3

Vorgehen

- ☒ Vervollständigen Sie den Namen + Link am Beginn des Dokuments
- ☒ Vervollständigen Sie die Wahrheitstabellen
- ☒ Beachten Sie die Limitierungen der Elemente!
- ☒ Exportieren Sie die Schaltung aus CircuitVerse und ersetzen Sie mit dem Export die Datei `./praktikum3.cv`. Beim späteren Veröffentlichen (Pushen) Ihrer Lösung auf Gitlab erhalten Sie unmittelbar Feedback, ob Ihre Schaltung funktional fehlerfrei ist.
- ☒ Laden Sie die Bilder aus CircuitVerse herunter und ersetzen die Dateien in `./images/` entsprechend. Beachten Sie die verschiedenen Schaltungs-Reiter, jede Schaltung ist separat. Laden Sie die PNG-Datei herunter wie im Wiki beschrieben.
- ☒ Überprüfen Sie, ob Ihre CircuitVerse Schaltungen den Richtlinen entsprechen: https://gitlab.lrz.de/grundlagen-der-rechnersysteme/semester/wise2026/hardware-teil/documentation/-/wikis/CircuitVerse

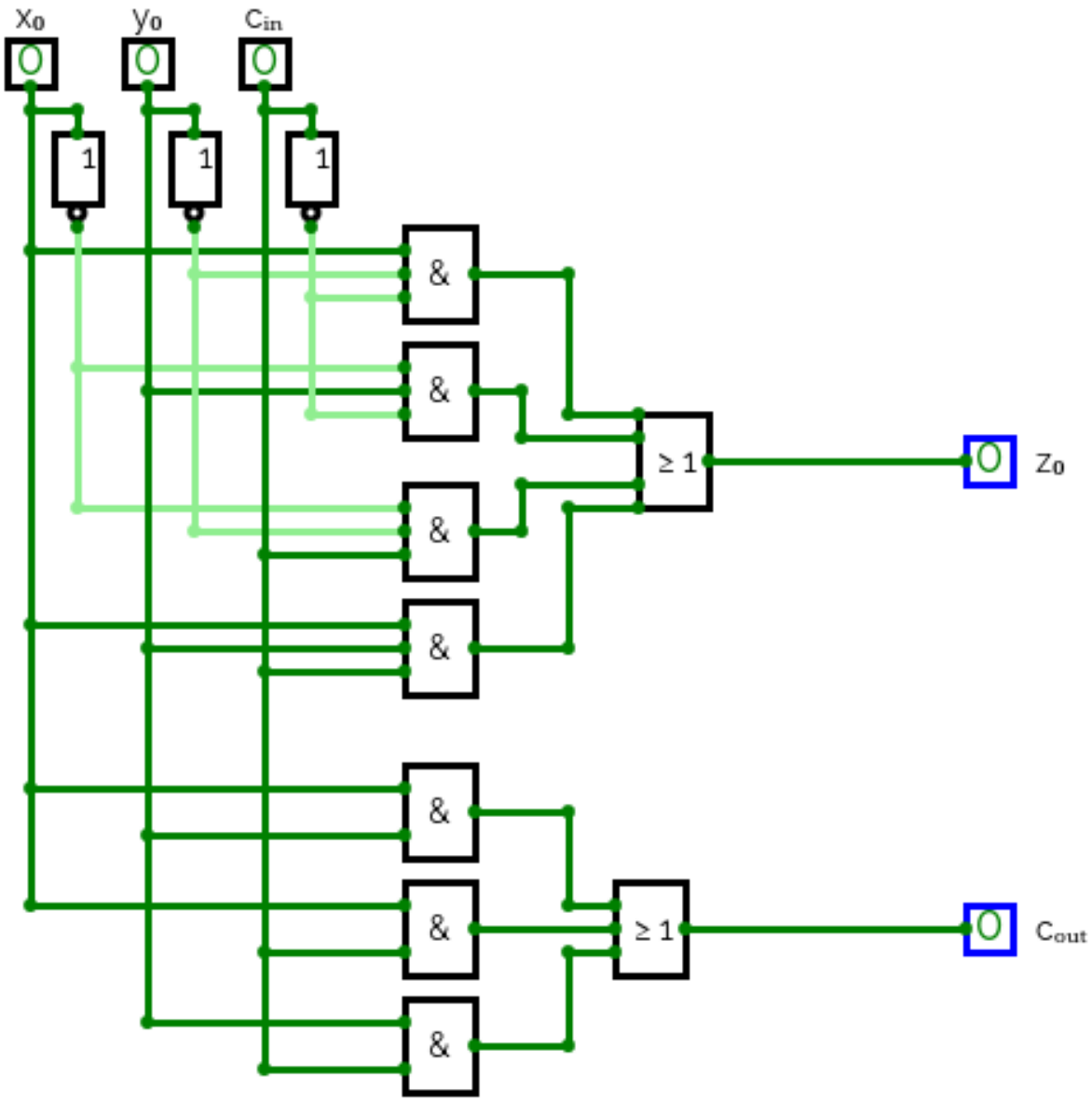
Aufgabe 1

a) Volladdierer als 2-stufiges Schaltnetz

Vervollständigen Sie folgende Wahrheitstabelle: Volladdierer

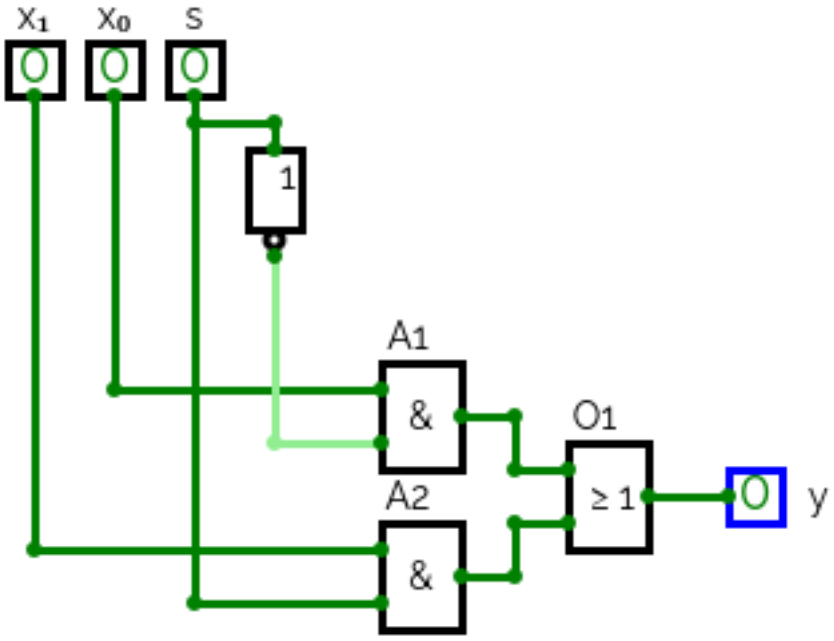
y_0	x_0	c_{in}	c_{out}	z_0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Schaltung in CircuitVerse: **Ersetzen Sie folgendes Bild durch Ihre Schaltung**

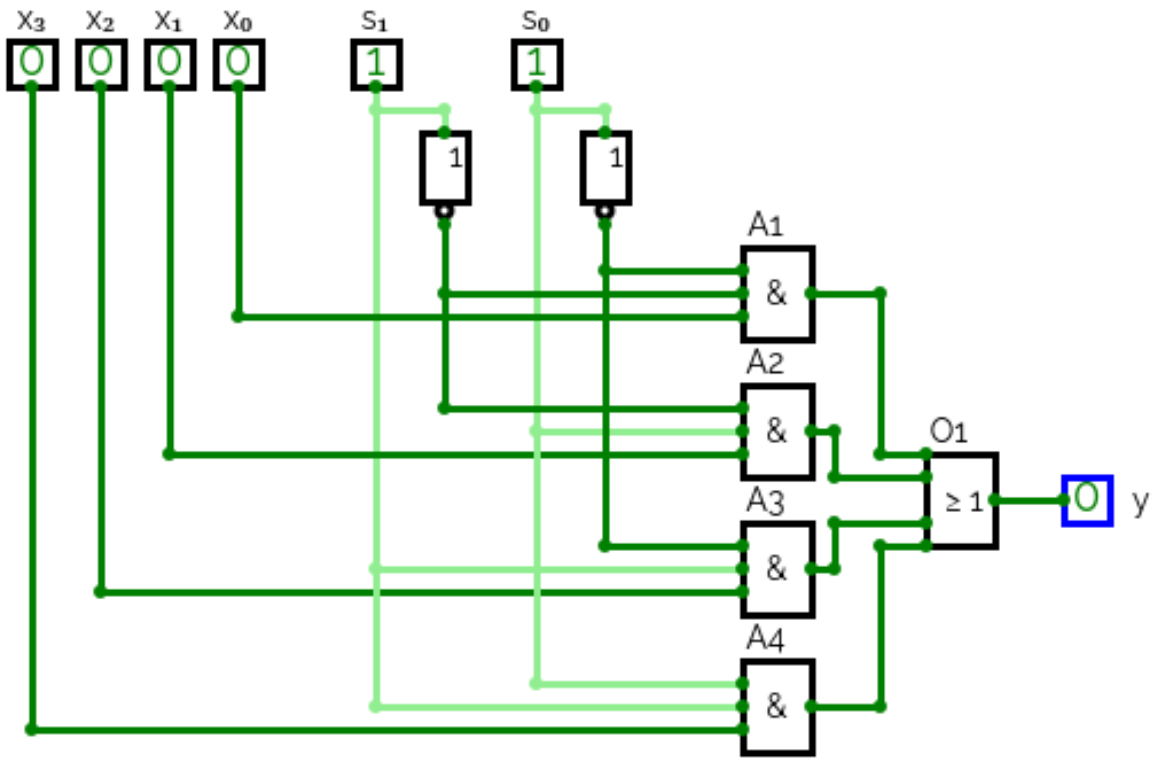


b) Multiplexer

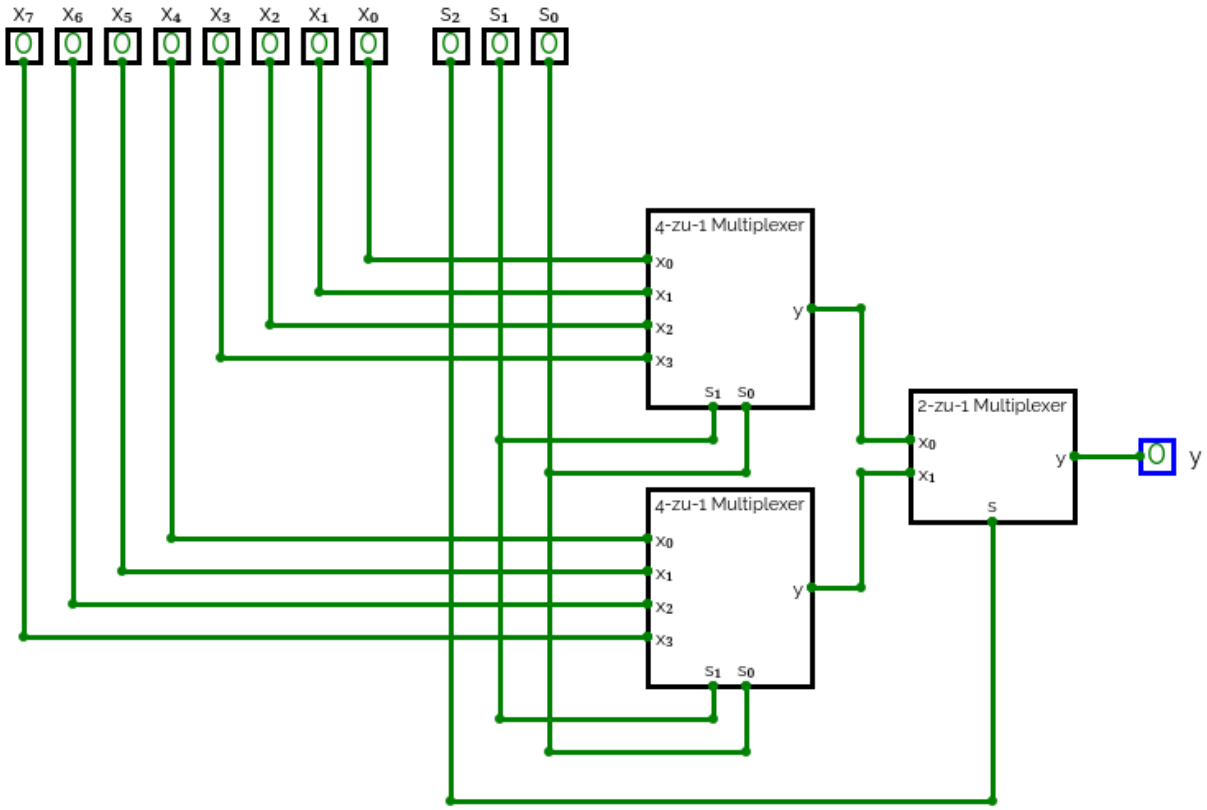
Schaltung in CircuitVerse: **Ersetzen Sie folgendes Bild durch Ihre Schaltung**



Schaltung in CircuitVerse: **Ersetzen Sie folgendes Bild durch Ihre Schaltung**



Schaltung in CircuitVerse: **Ersetzen Sie folgendes Bild durch Ihre Schaltung**



c) Zwei 2-bit Addierer mit drei 8:1 Multiplexer

Vervollständigen Sie folgende Wahrheitstabelle: $x_1x_0 + y_1y_0 = z_2z_1z_0$

x_1	x_0	y_1	y_0	z_2	z_1	z_0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

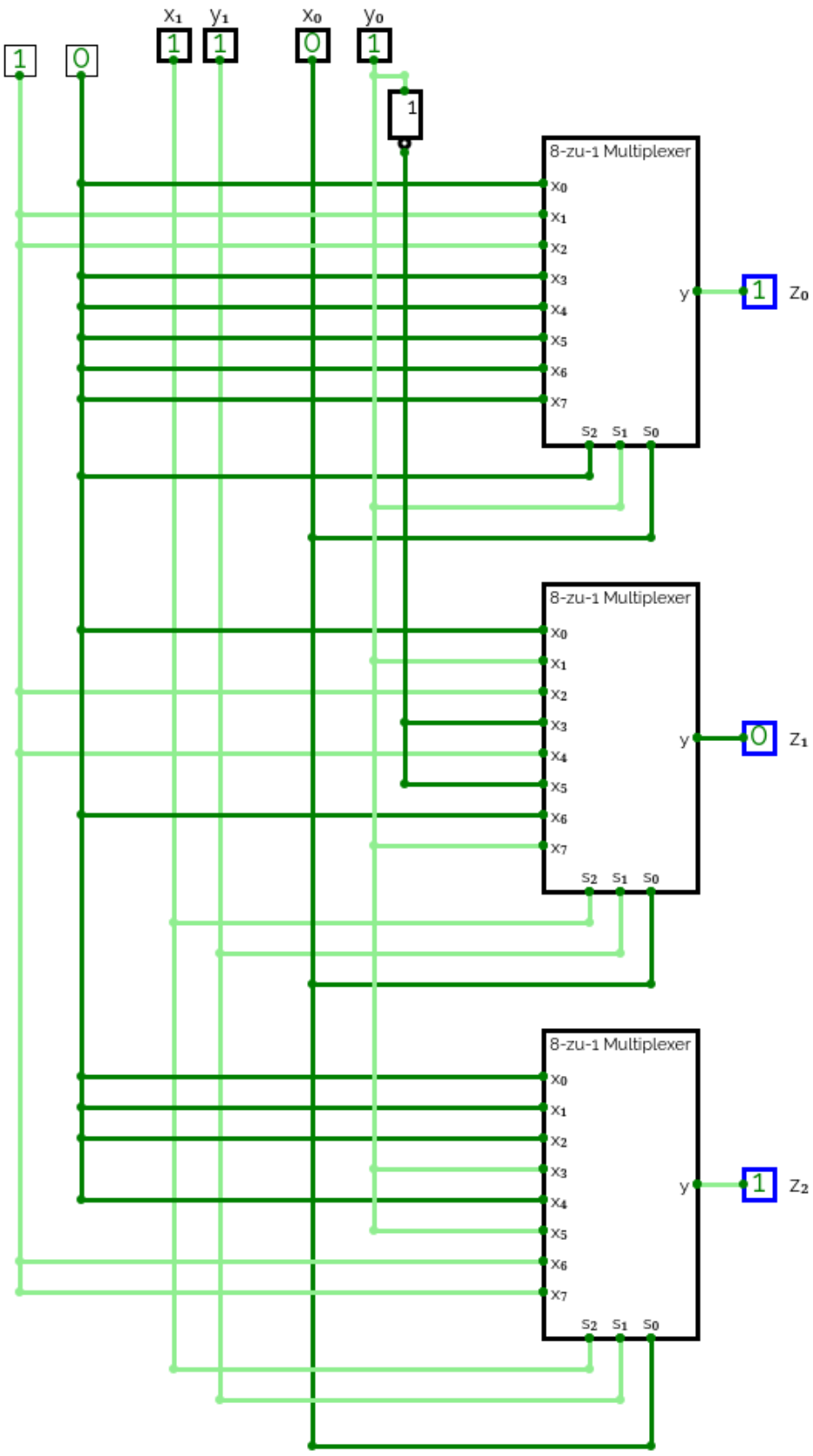
Für MX2

x_1	y_1	x_0	y_0	z_1	$y0 = z1$	
0	0	0	0	0	=	0
0	0	0	1	0	!	
0	0	1	0	0	=	y
0	0	1	1	1	=	
0	1	0	0	1	!	1
0	1	0	1	1	=	
0	1	1	0	1	!	noty
0	1	1	1	0	!	
1	0	0	0	1	!	1
1	0	0	1	1	=	
1	0	1	0	1	!	noty
1	0	1	1	0	!	
1	1	0	0	0	=	0
1	1	0	1	0	!	
1	1	1	0	0	=	y
1	1	1	1	1	=	

Für MX3

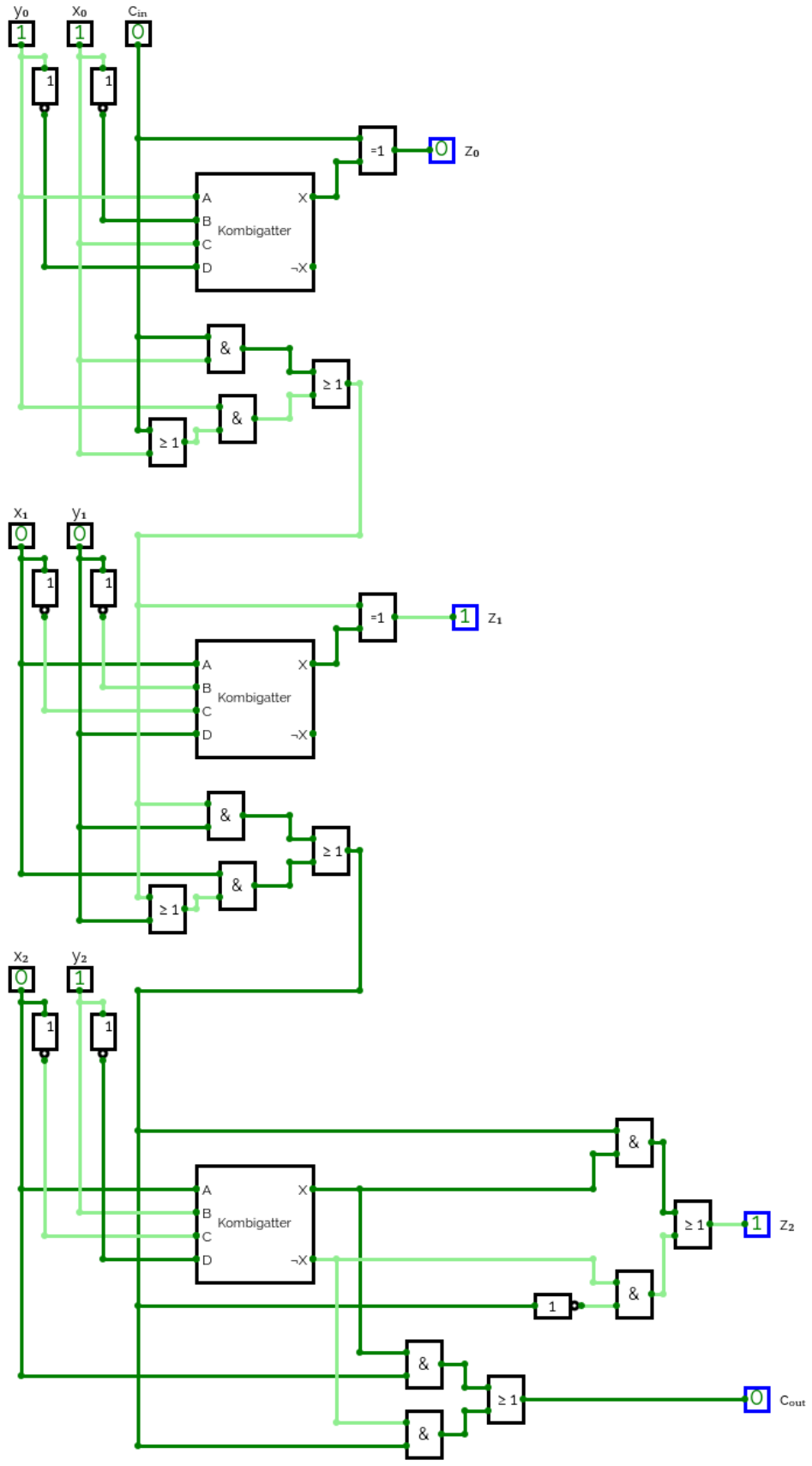
x_1	y_1	x_0	y_0	z_2	$y0 = z_2$	
0	0	0	0	0	=	0
0	0	0	1	0	!	
0	0	1	0	0	=	0
0	0	1	1	0	!	
0	1	0	0	0	=	0
0	1	0	1	0	!	
0	1	1	0	0	=	y
0	1	1	1	1	=	
1	0	0	0	0	=	0
1	0	0	1	0	!	
1	0	1	0	0	=	y
1	0	1	1	1	=	
1	1	0	0	1	!	1
1	1	0	1	1	=	
1	1	1	0	1	!	1
1	1	1	1	1	=	

Schaltung in CircuitVerse: **Ersetzen Sie folgendes Bild durch Ihre Schaltung**


x1
y1
x0
y0
1
0
1
1
0
1
1
8-zu-1 Multiplexer
x0
x1
x2
x3
x4
x5
x6
x7
s2
s1
s0
y
1
z0
8-zu-1 Multiplexer
x0
x1
x2
x3
x4
x5
x6
x7
s2
s1
s0
y
0
z1
8-zu-1 Multiplexer
x0
x1
x2
x3
x4
x5
x6
x7
s2
s1
s0
y
1
z2

Aufgabe 2: Ripple-Carry Addierer

Ersetzen Sie folgendes Bild mit Ihrer Lösung


y0
x0
cin
1
1
0
1
1
=1
0
z0
A
B
C
D
Kombigatter
X
¬X
&
≥ 1
&
≥ 1
x1
y1
0
0
1
1
=1
1
z1
A
B
C
D
Kombigatter
X
¬X
&
≥ 1
&
≥ 1
x2
y2
0
1
1
1
A
B
C
D
Kombigatter
X
¬X
&
&
≥ 1
1
z2
1
&
&
≥ 1
0
cout

Aufgabe 3: Dokumentation des geplanten Tests

Aufgabe: Dokumentieren Sie, wie Sie den Ripple-Carry-Addierer aus Teilaufgabe 2 testen werden.

1. Ripple Carrier auf dem Digiboard nachbauen
2. Eingänge x_2, x_1, x_0 auf dem Digiboard anstecken, dann möglicherweise mit einem Eingang abstand die anderen einbauen.
3. C_{in} kann egal sein, da er hier zu Beginn 0 ist und auf dem Digiboard beim ersten Gate im Volladdierer bzw hier Halbaddierer bei =1 einfach ignoriert wird. Falls gewünscht kann c_{in} als ein weiterer Input festgelegt werden.
4. Ausgänge an die 4 LEDs anstecken zum Visuellen überprüfen
5. Dann Beispiel rechnungen durchprobieren
   Option 1: Alle Rechnungen durchgehen
   Option 2: Um zu vermeiden für alle 7 Eingänge (2^7 Optionen) alles durchzuprobieren, eine Tabelle erstellen die alle Rechnungen testet. Hier werden jedoch manche Fehler vernachlässigt:
   Beispiel: Rechnung 111 + 001 -> Erfolgreich jedoch 001 + 111 hat einen Fehler der nicht erkannt wird:

Beispiel Tabelle für Testrechungen:

Nr.	$x_2x_1x_0$	$y_2y_1y_0$	Erwartet C_{out}	Erwartet $z_2z_1z_0$	Beobachtet	Was wird getestet
1	000	000	0	000		Alles 0
2	111	111	1	110		Alles 1, größte Summe
3	111	001	1	000		Carry läuft durch alle Stufen
4	001	111	1	000		Dasselbe mit vertauschten Operanden
5	001	001	0	010		Carry von Stufe 0 nach 1
6	011	011	0	110		Carry von Stufe 1 nach 2
7	100	100	1	000		Carry aus der letzten Stufe (C_out)
8	101	010	0	111		Abwechselnde Bits, kein Carry
9	010	101	0	111		Dasselbe vertauscht

6. Schauen ob die Licher alle richtig leuchten.
7. Wenn Fehler auftreten, systematisch Fehler finden durch testen der Halb und Volladdierer um Problem einzugrenzen
8. Abgeben